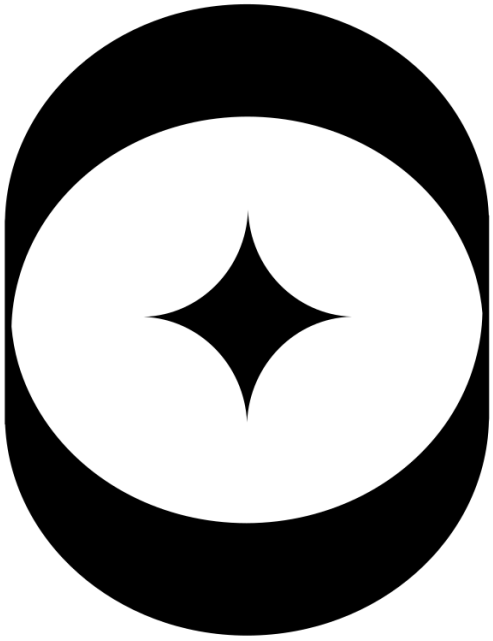

Ark Robotics

Yunhao Cao

Gidon Gautel

Gourav Mohanan

Skanda Nadig

1. Introduction and Background

This report details the start-up thesis and technical background of Ark Robotics (AR). At its core, AR is a company focused on autonomous inspection of crewed space stations and other high-value, high-risk space assets. We provide this service through a sensor-equipped CubeSat capable of autonomously navigating around a space structure, ensuring full sensor-coverage at acceptable fuel cost. We use nitrogen cold-gas thrusters to navigate our spacecraft, a commodity readily available and frequently resupplied to any crewed space station.

The key motivation for this service is the current exorbitant operational cost of space station inspection and monitoring. Astronauts currently spend between 40 and 140 hours per year conducting extravehicular activity (EVAs) on the International Space Station (ISS). The vast majority of these hours spent are for inspection and servicing purposes, detracting from value-added activities such as conducting science experiments. With an amortized cost of astronaut time of $100,00 per hour, and 2 or more astronauts conducting each EVA, this results in between $8m to $28m of direct cost spent on non-value-added activities per year, and an estimated $20m to $50m in total annual cost when accounting for opportunity cost (see Appendix 7.1). As the ISS is decommissioned and commercial space stations are launched to replace it, this problem will not go away. We will automate EVA activity to deliver cost-savings to NASA and commercial operators, allowing astronauts to focus all of their time on value-added activities, increasing operator ROI.

To address this problem, we specifically propose to provide the following service:

1. Full coverage path planning and autonomous execution of trajectories to fully sense space station exteriors
2. Autonomous detection of anomalies on the space station exterior

This report proceeds by reviewing related work, current and future competitors in Section 2. Section 3 provides a system overview, detailing the environment driving our requirements, our hardware, and our software and autonomy stack. Section 4 provides a more detailed analysis of our coverage path planning and controls approach, as well as providing a performance analysis of the current implementation. Section 6 concludes with areas for improvement.

2. Related Work and Market Research

2.1. Current Approaches

Space-station external inspection is currently conducted almost entirely by humans via EVA, however, is heavily assisted by an array of robotic systems. More specifically, Table 2.1 provides a full overview of these systems, and their specific role in supporting or partially replacing human EVA activity.

Robotic System	Description
Canadarm2	17m, 7DOF robotic arm capable of inch-worming around the ISS by relocating its base. Capable of camera-based visual checks via attached camera systems. However, largely designed for large-scale manipulation
Dextre	Two-armed, high-precision robotic manipulator attached to Cadadarm2. Offers fine motor skills e.g., tool use, connector handling, ORU replacement etc. Carries the Dextre pointing package and external cameras, enabling close-up high resolution inspection of components.
European Robotic Arm	11m arm capable of traversing the Russian segment. Carries free-flying end-effectors. Carries optical sensors and is capable of precision servicing of Russian modules, however, is also largely restricted to these modules.

Japanese Experiment Module Remote Manipulator System	Smaller arm with limited inspection capability. Capable of positioning external experiment cameras, e.g., to check the JEM Exposed Facility.
Robotic External Leak Locator	Purpose-built leak detection instrument deployed on Dextre. Carries mass spectrometer capable of tracing ammonia leaks from the station's cooling loops. We consider this complementary, rather than competitive to our system.
GITAI S2 Robotic Arm System	Next-generation dual-arm worker robot intended for servicing, assembly and inspection tasks. Carries high-resolution cameras. However, still fixed and surface mounted.

Table 2.1 Overview of Robotic Systems supporting and/or replacing EVA activity

These systems have several key drawbacks.

Firstly, and most importantly, *they do not allow full external sensor coverage of the ISS exterior*.

Secondly, they are costly to operate. For example, Canadarm2 is manually operated, usually by a team of ground-based operators. While no specific figures are publicly available, it is likely that the annual cost of this operating team runs in the millions of dollars.

Thirdly, while latency is not prohibitive, it is high enough to require careful planning, which slows down operations and increases costs.

2.2. Emerging Approaches

A number of emerging approaches exist that will compete with our solution.

2.2.1. Planned Systems for New Space Stations

Table 2.2 provides an overview of the planned systems for external inspection and servicing of emerging space stations.

Entity	**Planned Systems**
Genesis Engineering	Have proposed a pressurized free-flying spacecraft to allow astronauts to perform EVA-equivalent work without wearing a spacesuit. May allow for reduced inspection time, but likely would not reduce cost to a substantial degree, given the primary cost driver is labor costs for the astronauts themselves.
GITAI	GITAI has hinted at robotic crawlers that are able to move along space station trusses. No major details are available, however, this would allow for more versatile inspection capability than is currently available.

Table 2.2 Planner systems on future space stations

2.2.2. Horizontal Integration Risk

In addition to the planned systems detailed above, a number of companies currently exist in the market that have not explicitly stated plans to move into our vertical, but possess capabilities that could make them competitors in future. Table 2.3 provides an overview of these companies and their capabilities. Most companies are on-orbit servicing companies. While (to the knowledge of our team) they have not made public announcements regarding space-station inspection, their capabilities are aligned with this application, so they pose a non-negligible competitive risk.

Company	**Planned Capabilities**
ClearSpace	Offer debris removal and on-orbit servicing services. Sensing payload suite would be able to offer equivalent capability, but platform is likely overengineered for our application. Also uses non-competitive fuel for

	these purposes (hydrazine, which is toxic, and is not regularly resupplied to the ISS, or any future crewed station)
AstroScale	Essentially equivalent to ClearSpace. Fuel choice is the same (Hydrazine), so again, we consider them over-engineered and unsafe for our proposed niche.
Starfish Space	Offer inspection and life-extension services for spacecraft. Use electric propulsion, which does not offer sufficient thrust to achieve our mission profile.
Northrop Grumman	Similarly offers mission extension services. However, again, propulsion is an electric and hydrazine hybrid. Non-compatible with crewed space stations and our delta-v profile.

Table 2.3 Potential competitors and their capabilities

2.2.3. Academic Work

The most relevant paper to our work is that produced by Apodeca et al. [1]. For full disclosure, we took inspiration from the high-level approach of this work. However, our work differs along several key dimensions, some boosting system performance, and some differing:

2.2.3.1. Positive Differentiators

- Our processing pipeline allows for *any* model to be fed into the pipeline and a path to be produced
- Our pre-processing algorithm offers high coverage and acceptable time bounds with fuel costs that are within the capabilities of launchable platforms, not matching the academic literature on fuel use, but offering competitive performance on execution time and coverage

2.2.3.2. To Be Improved

- While our pipeline can take in any 3D model of a station, it does not produce a convex mesh, and therefore may not guarantee conditions relating to collision
- We do not currently use Clohessy-Wiltshire dynamics in our trajectory generation step. This reduces dynamic realism partially. However, given the modular design of our pipeline, swapping in more sophisticated dynamics models in future should be trivial
- We currently use a NN TSP solver in order to order our viewpoints, This may not produce the optimal path, though is reliable for our purposes. Again, this can be easily mitigated by swapping in an alternative software module

3. System Overview

3.1. Assumptions of Operating Environment & Limitations of Deployment

The requirements for our system were driven by the unique environment posed by our use case. Specifically, our system would need the capabilities to:

- Navigate completely and safely the exterior of 3D structures
- Maneuver accurately in a zero-G environment, in a fuel efficient manner
- Use a fuel that is readily available and safe in a crewed high-risk environment
- Have sensors suited for the external inspection of space structures, generating data of high enough fidelity to detect anomalies

3.2. Summary of Hardware

Driven by the environmental requirements and constraints, we settled on the following hardware configuration for our platform:

- A 6U CubeSat, capable of 6 DOF through the use of reaction wheels and thrusters

- Nitrogen cold gas thrusters. Nitrogen is readily available on any crewed space station and is regularly resupplied. It is non-explosive and non-flammable, and while posing an asphyxiation risk, protocols are already in place on crewed stations to manage this risk.
- Visual sensors: possible anomalies on the space station exterior include micrometeor impacts, arcing on solar panels, and other visible surface damage. Anomalies should therefore be detectable through an optical sensor suite in the visual band. This said, we are flexible to accommodate additional sensors if needed.
- High-accuracy navigation: We aim to use differential GNSS for positioning relative to the space station (requiring some custom work on a robust positioning system for use in LEO), as well as an IMU, and potentially beacons placed on the space station exterior, should positioning accuracy be insufficient.

We will use a lightweight and standardized cubic form-factor for our platform to reduce launch and material costs.

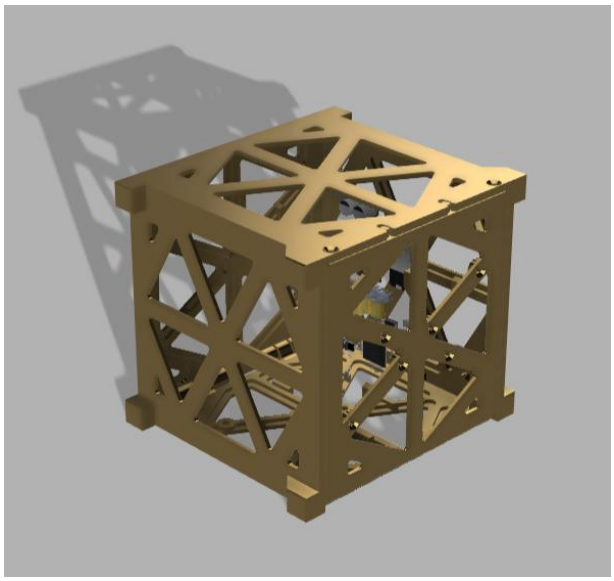

Figure 3.1 Mechanical prototype of structural primitives of the CubeSat

Figure 3.2 provides an exploded view of our complete hardware platform: A 6U CubeSat equipped with the necessary structural, power and sensor suite to execute the required mission.

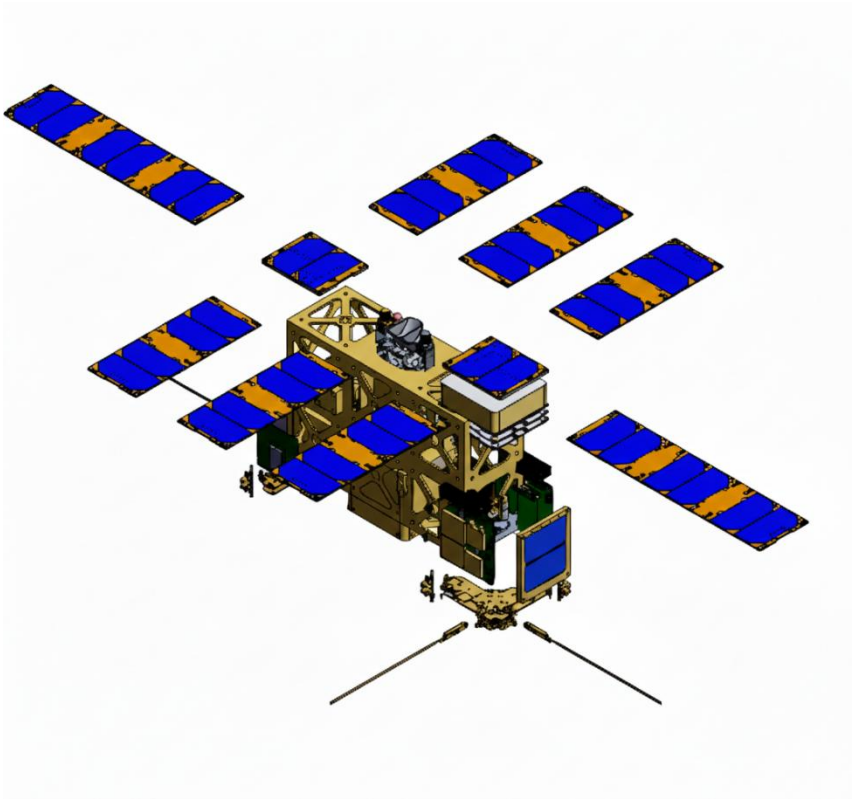

Figure 3.2 Hardware configuration of platform

3.3. Summary of Algorithms & Autonomy Stack

The below provides a high-level summary of the core components of our algorithm and autonomy stack. Further details, and analysis of their effectiveness are provided in Section 4.

The software stack is modular which means that each sub-system is its own module running efficient algorithms. The modules are monitored by the master module that controls the execution of modules based on the information set in the configuration file. This modular design makes the system flexible to use and highly customizable. Customization is one of the core requirements for the application of inspection robots and the method of customization has to be kept simple. We have achieved this by designing robust APIs and adaptable modules that enable configuration of the entire system with a single configuration file. The parameters to be added, modified and tuned are all designed and those that may need implementation in the future can also be introduced into the system seamlessly.

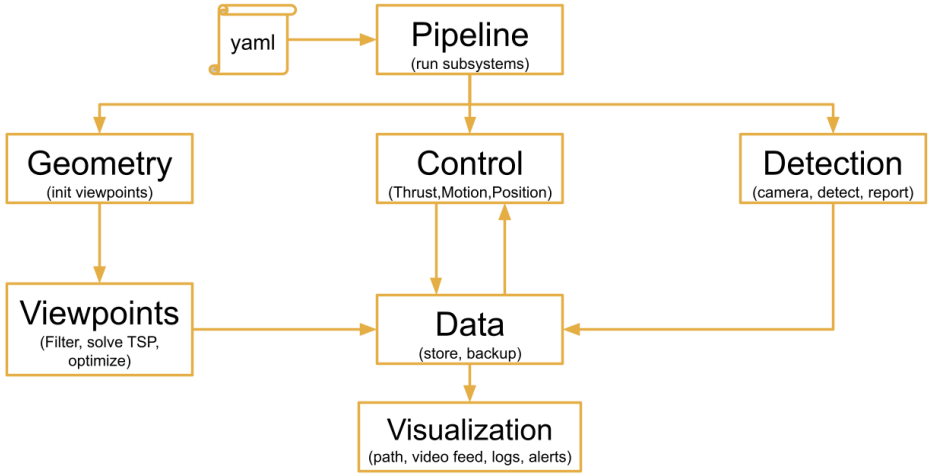

Figure 3.3 Software System Architecture

3.3.1. Coverage Path Planning Algorithm

Our coverage path-planning draws from the high-level approach taken by Apodaca et al. [1], however, has been adapted to our specific use case. Key differences are pointed out throughout the text in Section 4.1. On a high level, the path-planning algorithm takes the following approach:

1. The station model is pre-processed into a watertight triangular mesh of appropriate size for the path planning task
2. For each triangular face, a viewpoint is generated by projecting a point at a set distance from the normal of the face
3. Viewpoints are then reduced in number using a K-means clustering approach
4. The viewpoints are passed to a nearest neighbor travelling salesman planner, which produces an ordered set of viewpoints
5. Collisions in straight-line paths between viewpoints are resolved via a heuristic solver
6. For each viewpoint a pointing angle is produced by producing a pointing vector to the closest face
7. The viewpoints are passed to an optimizer which produces trajectories between each viewpoint, and may slightly shift the viewpoint within a given tolerance. The optimizer seeks to produce delta-v impulses at each viewpoint that optimize the time to traverse the entire path, and the fuel consumed for the entirety of the trajectory

The path planner is discussed in more detail in Section 4.1.

3.3.2. Control Algorithm

The path planner produces a path whereby the spacecrafts performs single impulses at each viewpoint that allow it to travel to the next viewpoint. However, for simulating our path in Mujoco, a PID controller was implemented, for several reasons:

1. A PID controller with continuous control inputs to the thrusters allows for dwell time at viewpoints
2. The PID controller allows for course correction in the case of external perturbations
3. The traversal time using a PID controller is faster (at the expense of fuel use)
4. The primary purpose of the simulation was to confirm that a spacecraft could safely navigate the path without compromising the safety of the station

The implemented controller is described in detail in Section 4.2.

3.3.3. Anomaly Detection

The Anomaly Detection subsystem provides real-time identification of cracks and surface defects using a monocular optical camera and a lightweight deep learning model. The system captures continuous video frames using OpenCV, preprocesses them for consistency, and performs inference using a GPU-accelerated convolutional neural network optimized for low-latency deployment. Detected anomalies such as cracks, surface damages, or small structural discontinuities are localized using bounding boxes and confidence scores, enabling reliable, on-the-fly assessment of structural health.

To maintain real-time performance, the subsystem integrates frame-level optimizations including down-sampling, asynchronous GPU inference, and selective processing. This design enables rapid anomaly highlighting without compromising detection accuracy, making the subsystem suitable for embedded platforms, robotics applications, and autonomous inspection systems.

4. Technical Contributions

4.1. Path Planning Pipeline

4.1.1. 3D Model Pre-Processing

The first step of the path planning pipeline is to take in a raw 3D model of a space station (or, for that matter, any inspection object of interest), and produce a watertight mesh with a reasonable number of faces that is computationally processable.

We utilized Blender Python scripting in order to allow for repeatability and controllable parameterization of mesh processing. A model of the ISS provided by NASA[1] was fed through the pipeline and the following transformations were applied:

1. **Mesh Joining:** In the case of the ISS model, the model is composed of a collection of smaller objects. These are joined into a single mesh. This unprocessed single mesh is exported for use in our MuJoCo simulation.
2. **Voxel Remesh:** This step ensures that the final mesh is a single, watertight object. The station object is disaggregated into a 3D grid of cubes (voxels), from which an approximate surface is extracted from the occupancy grid. This results in a watertight mesh with no holes or manifold edges. This is useful for later path planning stages, as it prevents the generation of spurious viewpoints, and simplifies collision detection.
3. **Laplacian Smoothing:** Given the fact that voxel remeshing tends to produce irregular or "blocky" surfaces, a Laplacian smoothing step was added to smooth the mesh by moving vertices towards the average of their neighbors.
4. **Decimation:** Blender's decimate modifier seeks to collapse edges and remove faces that contribute the least to visual shape. This step was added to reduce the total face count and make the model more computationally processable.
5. **Weld Vertices:** Similar to the previous step, welding vertices combines vertices that are within a chosen threshold distance. This step removes duplicate or redundant vertices, cleans up "cracks" and "gaps", and ensures the mesh is topologically palatable for later geometric operations.
6. **Triangulation:** Later path planning steps count on a triangulated mesh. The resulting mesh of prior steps is therefore transformed to be composed of purely triangular elements.

Each of these steps are tunable via the following variables, implemented in the model pre-processing step:

- Voxel Size

---

[1] NASA. 2023. International Space Station 3D Model. Available Online: https://science.nasa.gov/resource/international-space-station-3d-model/ [last accessed: 11/28/2025]

- Smoothing iterations
- Laplacian smoothing factor
- Decimation ratio
- Weld threshold

These parameters were tuned manually until a smooth mesh with sufficient faces to produce meaningful paths, but few enough faces to be computationally feasible on our machines was achieved.

Even with this repeatable processing pipeline, some features, such as very thin solar panels, or thin, geometrically complex trusses, proved too difficult to process in an automated fashion. Therefore, these features were implemented manually in the Blender GUI in the form of singular, voluminous features superimposed on the original mesh. This new mesh was passed through the processing pipeline. A visual flow of the steps for model pre-processing is provided in Figure 4.1.

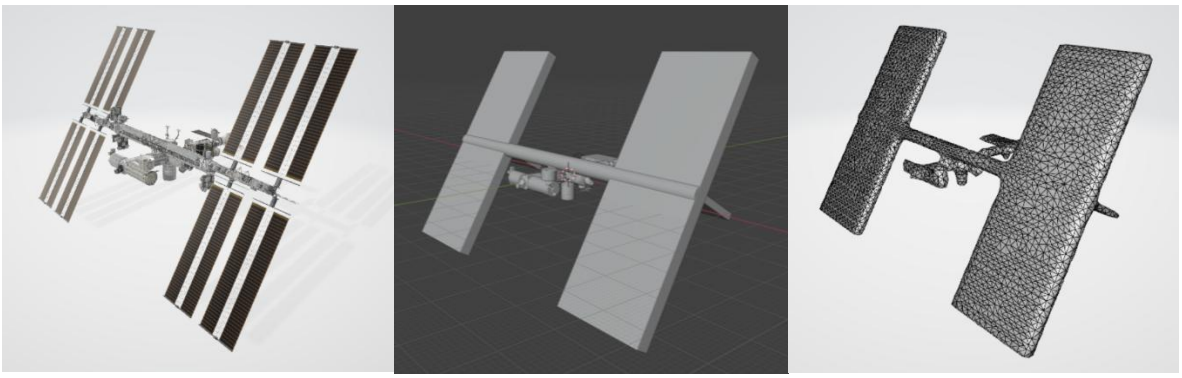

Figure 4.1 Visualization of pre-processing steps

4.1.2. *Viewpoint Generation & Clustering*

The viewpoint generation step seeks to produce candidate camera positions that are well-aligned with the inspection object. We have implemented two different methods of viewpoint generation.

4.1.2.1. Fibonacci Sphere Method

To ensure uniform and unbiased coverage of a 3D object during inspection, our system employs the Fibonacci Sphere method to generate a set of evenly distributed viewpoints around the object. This technique is widely used in graphics, robotics, and sampling applications due to its ability to approximate uniform point distributions on a sphere with far less clustering than latitude–longitude or random sampling methods.

4.1.2.2. Method Overview

The Fibonacci Sphere algorithm leverages properties of the golden angle to generate points on a sphere that naturally avoid clustering. For a desired number of viewpoints N, each point is defined by incrementally stepping around the sphere at an angular separation proportional to the golden angle.

4.1.2.3. Golden Angle

The golden angle Φ derives from the golden ratio $\varphi = \frac{1+\sqrt{5}}{2}$, and is defined as:

$$\Phi = \frac{2\pi}{\varphi^2} \approx 2.399963229\, rad$$

4.1.2.4. Algorithm to Generate the Sphere of Viewpoints

For each index $i \in \{0,1,\dots,N-1\}$, the spherical coordinates are constructed as:

$$y_i = 1 - \frac{2i}{N-1}$$

$$r_i = \sqrt{1 - y_i^2}$$

$$\theta_i = i \cdot \Phi$$

The corresponding cartesian coordinates become:

$$x_i = r_i cos(\theta_i)$$

$$z_i = r_i sin(\theta_i)$$

Each point (x_i, y, z_i) forms one camera viewpoint.

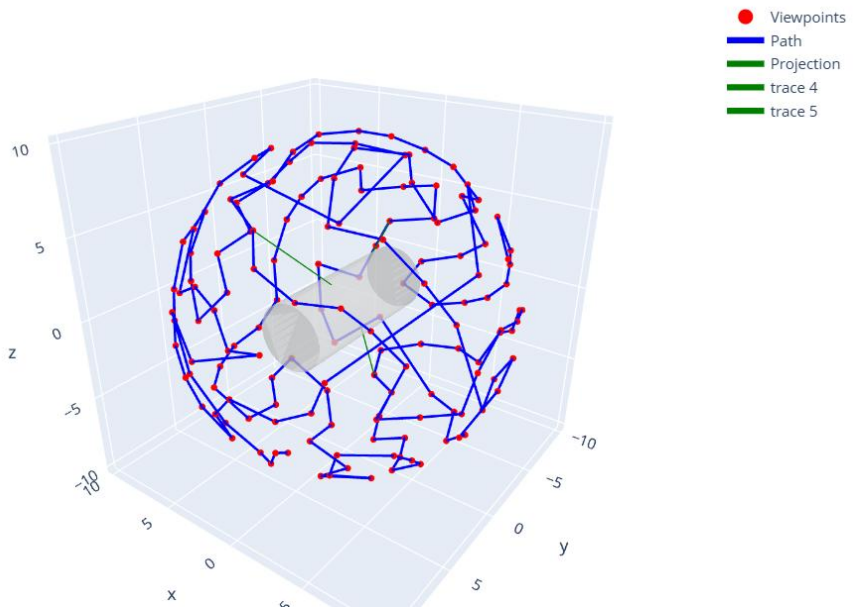

Figure 4.2 Point generation around a 3D model using Fibonacci Sphere Method

4.1.2.5. Ray-casting Method

Viewpoint generation is implemented by stepping away from each triangular face's centroid along the surface normal. Concretely:

Centroids are calculated from each triangle's three vertices v_{i1}, v_{i2}, v_{i3} as:

$$c_i = \frac{v_{i1} + v_{i2} + v_{i3}}{3}$$

The normal is given by:

$$n_i = \frac{(v_{i2} - v_{i1}) \times (v_{i3} - v_{i1})}{\|(v_{i2} - v_{i1}) \times (v_{i3} - v_{i1})\|}$$

Normals are then normalized, which allows for viewpoints to be offset by an arbitrary distance, which is parameterized within our pipeline.

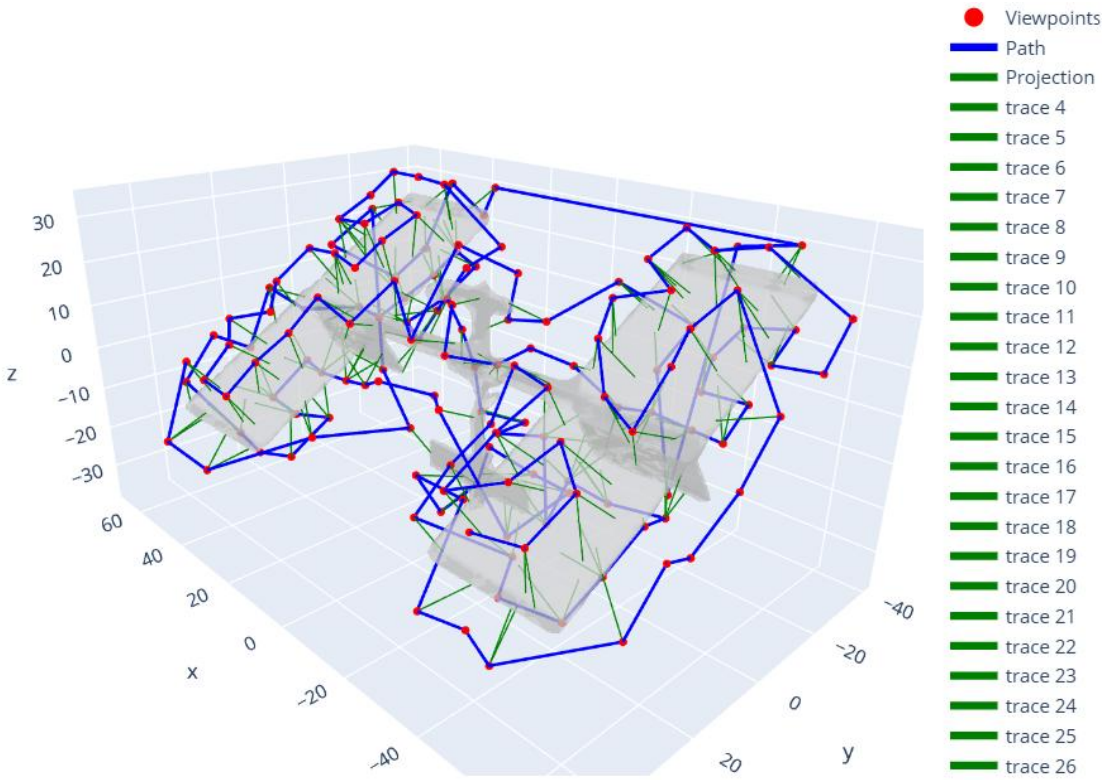

Figure 4.3 Point generation around the space station model using Ray-casting method

Because of the previous pre-processing step, our model is watertight, which ensures that normals are consistently outwards from the station body. However, even with the face-reduction steps taken previously, this can still result in thousands of viewpoints, which is impractical. Therefore, a clustering step is added to reduce the number of viewpoints, while maintaining information gain and coverage.

Two clustering approaches of viewpoints were implemented, configurable via our config files in the planning pipeline:

1. **DBSCAN:** Cluster viewpoints according to the density of the viewpoint cloud**.** DBSCAN is a density-based clustering algorithm that groups together points that are closely packed, marking points in low-density regions as outliers. It works by finding core points (those with a minimum number of neighbors within a set radius, epsilon) and expanding clusters from them, which allows it to find clusters of arbitrary shapes and handle noise effectively
2. **K-Means:** We use the unsupervised machine learning algorithm to partition filtered viewpoints into k distinct, non-overlapping groups (clusters), aiming to make viewpoints within a cluster similar, represented by centroids (cluster means). It's an iterative process that starts by randomly picking k centroids, assigns data points to the nearest centroid, then re-calculates the centroid as the mean of its assigned points, repeating until centroids stabilize. Its goal is to minimize the within-cluster sum of squares (WCSS), creating compact, isolated clusters.
    a. Compute k-means clusters
    $$C = \{C_1, \dots , C_k\}$$
    b. Select the centroid of each cluster as the representative viewpoint
    $$c_i = \frac{1}{|c_i|} \sum_{v \in C_i} v$$
    c. Feed the K cluster centers into the TSP solver for viewpoint ordering

4.1.3. Viewpoint ordering

We formulate the viewpoint ordering problem as a Traveling Salesman Problem (TSP), where each viewpoint corresponds to a "city," and the objective is to find the minimum-cost tour visiting all viewpoints exactly once. We employ a lightweight TSP solver suitable for robotics, where computational efficiency is essential for real-time or near real-time mission planning.

4.1.3.1. Problem Setup

Given a set of N filtered viewpoints after clustering:

$$V = \{v_1, v_2, \dots , v_n\}, v_i \in \mathbb{R}^3$$

we define the cost of traveling between viewpoints v_i and v_j as the Euclidean distance:

$$d(v_i, v_j) = \|v_i - v_j\|_2$$

The objective is to find a permutation in range {1,...,N} that minimizes the total traversal distance:

$$L(\pi) = \sum_{k=1}^{N-1} d(v_{\pi(k)}, v_{\pi(k+1)})$$

This corresponds to a classical symmetric TSP with metric distances.

4.1.4. Collision Avoidance

The straight line paths between ordered viewpoints are not guaranteed to be collision free. To mitigate this, we employ a heuristic collision avoidance system that "repairs" the set of viewpoints $\{k_i\}$ by introducing intermediate waypoints that result in a safe path. The result is a set of waypoints $\{w_i\}$ that produce safe coverage.

The station center is first approximated as the mean of all face centroids. This provides a simple reference point for later calculations.

For each pair of viewpoints, the system samples a parametric number of points along the straight line path between points. We compute relative vectors between sampled points and face centroids. We then compute an outward unit normal from the face, and calculate the signed distances to each face plane. A sampled point is then safe if, for any plane, the signed distance is positive and greater than a given margin.

To repair paths with detected collision, we produce waypoints along tangent directions of the straight-line path between viewpoints at the midpoint of the path. Candidate waypoints are produced along four perpendicular tangential directions at set distances, and checked for collision. The waypoint closest to the starting position of the path is chosen. This process is repeated recursively until either the path is safe or the recursion depth (specified in our yaml file) is reached. If no safe waypoint is found, a warning is logged, which requires manual repair.

Figure 4.4 provides an example of the collision avoidance heuristic in action for a simplified "station". As can be seen, a safe intermediate waypoint is successfully found for a straight-line path that is very obviously a collision with the station.

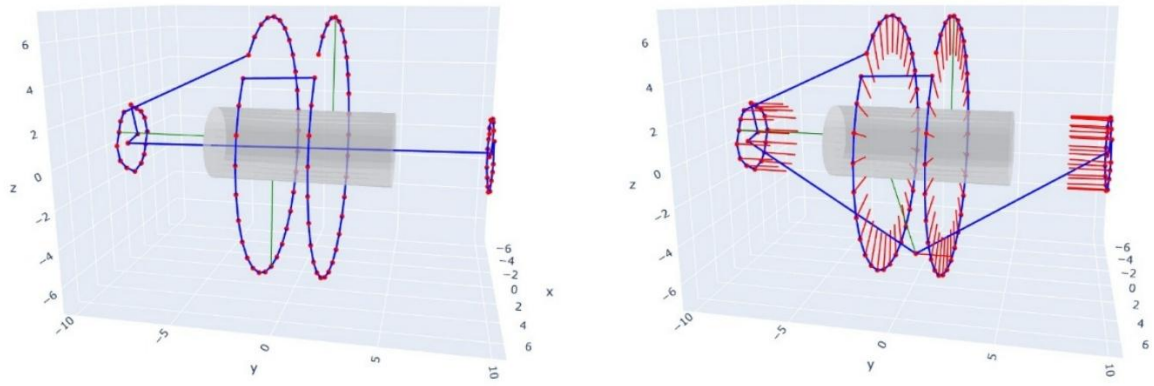

Figure 4.4 Simplified example of the collision avoidance system in action

4.1.5. Pointing Angle Generation

Given an ordered sequence of waypoints around the station, producing corresponding pointing vectors at each waypoint is comparatively simple. Our solution takes in the set of collision free waypoints, as well as the face centroids of the station mesh.

Viewpoints are first classified as real waypoints or intermediate waypoints generated through the collision avoidance process. For real waypoints, the system picks the nearest mesh face centroid and produces a vector pointing at it, by:

1. Computing the distance from waypoint w_i to all centroids
2. Selecting the closest face
3. Producing a normalized vector from the waypoint to the centroid of that face

For intermediate waypoints, the pointing angle of the prior waypoint is simply copied.

When pointing angles are passed to the control system of our stack, their respective orientations are converted into quaternion format for simple processing by MuJoCo.

4.1.6. Trajectory Generation & Optimization

At this stage of our path-planning system, we have a set of collision free ordered waypoints, however, the challenge remains to produce a dynamically feasible trajectory that visits each one. One solution is to apply an impulsive maneuver at each waypoint, drifting along the intermediate distance. Impulsive maneuvers can be characterized in terms of the directional delta-v required at each viewpoint, given the dynamic state from the prior maneuver. We want to do this in a manner that minimizes fuel and time cost while performing at a level realistic to the physical system.

The problem statement for trajectory generation is therefore: Produce the set of delta-v's to reach each viewpoint, minimizing fuel and time cost, subject to viewpoint tolerance, delta-v maximum magnitude, leg-time bounds. We also constrain the speed upon reaching the final viewpoint to be close to zero.

To solve this optimization problem, we employ a simple "free-floater" dynamics model, whereby the spacecraft state is described by:

$$x = \begin{bmatrix} r \\ v \end{bmatrix}$$

Where r is position and v is velocity. The continuous dynamics between impulses are then described by:

$$\dot{r}(t) = v(t), \qquad \dot{v(t)} = 0$$

At the beginning of each leg between waypoints, k, an impulsive velocity change Δv_k is applied for an instantaneous update:

$$v_k^+ = v_k + \Delta v_k$$

And over the leg duration, Δt_k, the state simply evolves as constant-velocity drift:

$$r_{k+1} = r_k + v_k^+ \Delta t_k$$

For each leg, k, therefore, the optimizer chooses impulse Δv_k and leg duration Δt_k, according to the objective function and constraints. The decision vector for optimization is therefore a stack of delta-v's and leg times:

$$x = \begin{bmatrix} \Delta v_0 \\ \Delta v_1 \\ \cdots \\ \Delta v_{n_{legs}-1} \\ \Delta t_0 \\ \Delta t_1 \\ \cdots \\ \Delta t_{n_{legs}-1} \end{bmatrix}$$

The objective function is then the weighted sum of fuel usage (using squared impulse as a proxy) against total time:

$$J(x) = \sum_{k=0}^{n_{legs}-1} \left(w_{fuel} \|\Delta v_k\|_2^2 + w_{time} \Delta t_k \right)$$

For our base case, w_{fuel} was chosen as 1, and w_{time} as 0.1.

As mentioned, three constraints, implemented to be in the form $g(x) \leq 0$, are applied to every leg:

- The end of every leg must lie within a radius around the second waypoint (which can be determined using our dynamics model)
- The delta-v at each step cannot exceed a given value
- The final speed on the last leg cannot exceed a given value, close to zero
- The leg duration cannot exceed an upper bound

In code, we used CasADi to formulate the optimization problem, which, summarizing the above, is composed of:

- Our decision vector:$x = \begin{bmatrix} all\ \Delta v_k \\ all\ \Delta t_k \end{bmatrix}$
- Our objective: $f(x) = J(x)$
- And our constraints $g(x) \leq 0$

We used the CasADi nlpsol (non-linear program solver) for the in-code implementation.

Finally, the output of the optimization step is the optimal delta-v, Δv_k^*, for each leg, and the optimal time, Δvt_k^*, for each leg. These can then be easily processed into a format which is executable for a controller set-up to execute impulsive maneuvers.

In the end, a set of waypoints positions, arrival velocities, and quaternions is exported for direct input into a controller.

4.2. Spacecraft Control

For simulation purposes, we employed a PD controller to achieve the given waypoints and pointing angles, rather than a controller that executes the impulsive maneuvers produced in the optimization section above. This was done for practical reasons, allowing us to view trajectory execution in real time, as well as for the reasons listed in the section below. PD control will inevitably lead to increased fuel

use, so for a real-life implementation, impulsive control of the trajectory produced through optimization would be the chosen path.

4.2.1. Description

This section describes the low-level controller used to track viewpoint positions and attitude for our space robot. The controller is intentionally lightweight and robust. It is deployed on a 29.5 kg platform equipped with six face-mounted thrusters (each capped at 20 N) and three reaction wheels (each capped at 5 N·m). PD control is adopted for three main reasons:

1. Simplicity and robust performance: PD requires few parameters and avoids online optimization.
2. Simple Dynamics: Each axis is shaped as an independent second-order system, enabling easy modeling.
3. No integral term by design: Under the assumption of negligible constant biases (well-aligned thrusters and wheels), steady-state errors are small enough to be ignored. This also eliminates integral windup under actuator saturation—an issue that does not arise with pure PD.

The control objective is to (i) track position/velocity in world frame using thrusters; (ii) track attitude/angular velocity using reaction wheels, given a reference input vector

$$input = [p, v, \theta, w] \in \mathbb{R}^{3+3+4+3}$$

where $p, v \in \mathbb{R}^3$ are the desired position and velocity (world frame), $\theta \in \mathbb{R}^4$ is the desired unit quaternion representing attitude (world frame), and $w \in \mathbb{R}^{3+3+4+3}$ is the desired angular velocity (body frame).

4.2.2. Translational PD (Acceleration)

A critically damped second-order controller is designed independently for each axis. The position and velocity tracking errors are defined as

$$e_p = p - p^*$$

$$e_v = v - v^*$$

$$a_{des} = -k_p e_p - k_v e_v$$

where p* and v* are the reference position and velocity, a_{des} is the desired acceleration output. For each axis, the closed-loop dynamics are modeled as a standard second-order system

$\ddot{x} + \frac{k_v}{m}\dot{x} + \frac{k_p}{m}x = 0$, $\begin{cases} k_p = mw_n^2 \\ k_v = 2m\zeta w_n \end{cases}$, where for critical-damped, $\zeta = w_n = 1$

with $m = 25kg$, this yields $k_p = 25$ and $k_v = 50$.

4.2.3. Attitude PD (Quaternion Error)

Similar to the translational PD controller, we define the angular-velocity error as

$$e_w = w - w^*$$

where w^* is the reference angular velocity. The attitude error is first expressed as a quaternion error

$$e_q = q^* \odot q^{-1}$$

where q^* is the reference quaternion, $\odot$ denotes the quaternion (Hamilton) product, and q^{-1} is the conjugate of current quaternion. We then map the quaternion error to a 3-D attitude error vector via

$$e_R = 2q_v$$

where $q_v \in \mathbb{R}^3$ is the vector part of quaternion q. Finally, the PD type torque feedback law

$$\tau = -K_R e_R - K_w e_w$$

is implemented.

This PD controller is simulated in MuJoCo with a control frequency of 500 Hz. The burn time of each thruster is accumulated as

$$T = N\Delta t$$

where N is the number of simulation steps (frames) during which the thruster is active, and Δt is the time step of each frame, for the purpose of overall fuel consumption evaluation.

4.3. Hardware

4.3.1. Top-Level Requirements

At a high level, our requirements for the actual inspection platform were as follows:

- Use a safe fuel that is readily available and cheap
- Offer sufficient sensing capability to allow for the detection of surface-level anomalies
- Offer sufficient thrust and specific impulse to perform the mission
- Be sufficiently lightweight to not incur large launch costs
- Have sufficiently small dimensions to allow for launch through a standard airlock
- Offer a high level of positioning capability to accurately execute paths and avoid any safety issues

These requirements drove our hardware selection.

4.3.2. Structure

In order to achieve a lightweight structure, a standard CubeSat frame structure was utilized in order to reduce weight and material costs. Figure 4.5 shows the final structural design. This also allows for easy mounting of all internal and external components.

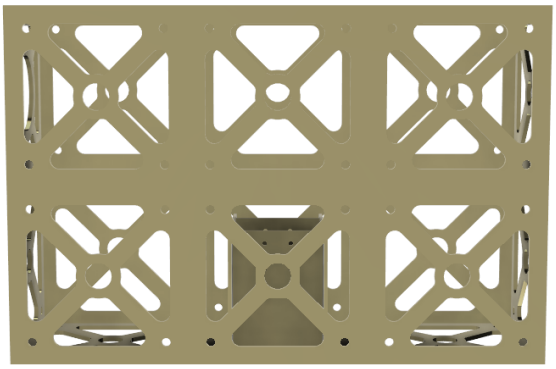

Figure 4.5 Final CubeSat structure design

4.3.3. ADCS

Our CubeSat requires 3 degrees of rotational freedom. To this end, we will employ three reaction wheels within the CubeSat structure. These will allow for attitude adjustment without the use of fuel.

For guidance, we intend to use a multimodal sensor suite. Specifically, we intend to employ differential GNSS, an onboard IMU, and potentially radio-beacons mounted on the station exterior as well as star trackers.

4.3.4. CDH

While our guidance algorithm is offline, the CubeSat will have to incur fairly heavy computational loads to process the data produced in imaging. To this end, we have provisionally selected the Xiphos

Q7 processor for command and data handling (CDH). This platform employs a Xilinx Zynq SoC, which includes ARM dual-core Cortex-A9 MPCore processors supported by programmable logic hardware.

4.3.5. Power & Propulsion

We suggest to use a combination of solar and battery power for our CubeSat's power. While the CubeSat could be charged by a space station's on-board power resources, this should not be relied on solely, therefore we have equipped our satellite with standardized solar panels.

As mentioned, we will use cold gas nitrogen thrusters for propulsion. Our baseline assumption is for thrusters with 70 Isp and approximately 50mN of thrust.

4.3.6. Sensing

We will use optical payloads for imaging purposes. Figure 4.6 shows the CAD model for the COTS imager selected for our initial CAD design.

Figure 4.6 render of imager used in CAD CubeSat design

4.4. Anomaly Detection

The Anomaly Detection subsystem is responsible for identifying cracks, dents, and other surface defects on metallic structures using an optical camera. For this proof-of-concept, we use a Logitech 1080p HD webcam to capture real-time video of the surface and apply a combination of classical computer vision (OpenCV) and deep learning-based segmentation to detect anomalies.

4.4.1. System Architecture

$$I_t \rightarrow F_t \rightarrow X_t \rightarrow M_t \rightarrow O_t$$

Where:

- I_t = raw image at time t
- F_t = filtered/clean frame
- X_t = normalized, resized model input
- M_t = model output mask (probability of crack/damage)
- O_t = final overlay visualization

The detection model returns bounding boxes and segmentation masks overlaid on the live video stream.

4.4.2. Pre-processing Pipeline

Before detection, the frame is enhanced to highlight small cracks:

1. Grayscale conversion
2. Contrast-Limited Adaptive Histogram Equalization (CLAHE)
3. Gaussian smoothing to reduce noise
4. Edge-enhancing filters (e.g., Sobel or Canny)

5. Edge Feature Extraction:

$$E_t = \nabla F_t = \sqrt{\left(\frac{\partial F_t}{\partial x}\right)^2 + \left(\frac{\partial F_t}{\partial y}\right)^2}$$

4.4.3. Crack Detection Model

We use a lightweight UNet-based semantic segmentation model trained or adapted for surface defect detection (e.g., MVTec AD or similar dataset). This model outputs a probability mask:

$$M_t(x, y) = P(crack \mid X_t(x, y))$$

A pixel is classified as a crack if:

$$M_t(x, y) > \tau, where\ the\ default\ value\ of\ \tau = 0.5$$

4.4.4. Post-processing and Bounding Box Extraction

The probability mask is thresholded, cleaned and converted into bounding boxes.

4.4.5. Experiments

The detection subsystem was tested independently in the real world on primarily metal surfaces to simulate the body of the space station. The best candidates for real world testing were domestic cars since the type of damage and frequency of occurrence favoured our requirements for testing. The anomaly detection system was evaluated on a custom dataset consisting of optical camera images of metal and composite surfaces with artificially introduced cracks, scratches, dents, and corrosion patterns. Data was collected using a 1080p USB webcam under controlled indoor lighting. The detection pipeline ran on a Lenovo Legion Y520 (Intel i7-7700HQ CPU, NVIDIA GTX 1050 Ti GPU) with Python 3.10, PyTorch, and OpenCV.

Figure 4.7 Example result of the crack detection system on a terrestrial vehicle

5. Results

5.1. Path Planning Results

Our pipeline is highly parameterized, with parameters within the modelling pre-processing and clustering steps resulting in significant variance in the resulting path and trajectory. We therefore present results for a base case with the following key set-up values:

Model Pre-Processing:

- Voxel size: 1.2
- Smoothing iterations: 5
- Smoothing lambda: 0.3
- Decimation ratio: 0.9
- Weld threshold: 1.3

Clustering:

- Method: kmeans
- Number of clusters: 150

5.1.1. Runtime Performance

- Nearest Neighbor runtime: 0.3-0.7ms
- The TSP solver was tested on filtered viewpoint sets ranging from 700 to 800 points.
- 2-opt refinement runtime: 5-15ms
- Total TSP optimization time: < 20ms, suitable for online mission planning.

5.1.2. Path Savings

The 2-opt refinement reduced path length by 18-32% compared with the naive Nearest Neighbour tour. Compared with random ordering, the optimized path achieved 50-65% reduction.

5.1.3. Coverage

We define a face as "viewed" if all three vertices are in direct line of sight of a particular viewpoint. Given our viewpoints, model mesh, and previously stored face information, it was therefore fairly simple to compute and visualize coverage within the pipeline.

For the ISS base case, we achieved a coverage of 99%. While exact figures for current coverage via the existing mounted robotic arms is not available, we believe this is far in excess of the current state of the art. In particular, inspection of solar panels and radiators currently requires astronaut EVAs. Our free floating platform inspects these, as well as all habitation modules, in a single sweep. Figure 5.1 shows the missed-coverage plot for the base case. Missed faces are marked with a red dot. As can be seen, only a single face is missed in this case, which results in the extremely high coverage observed.

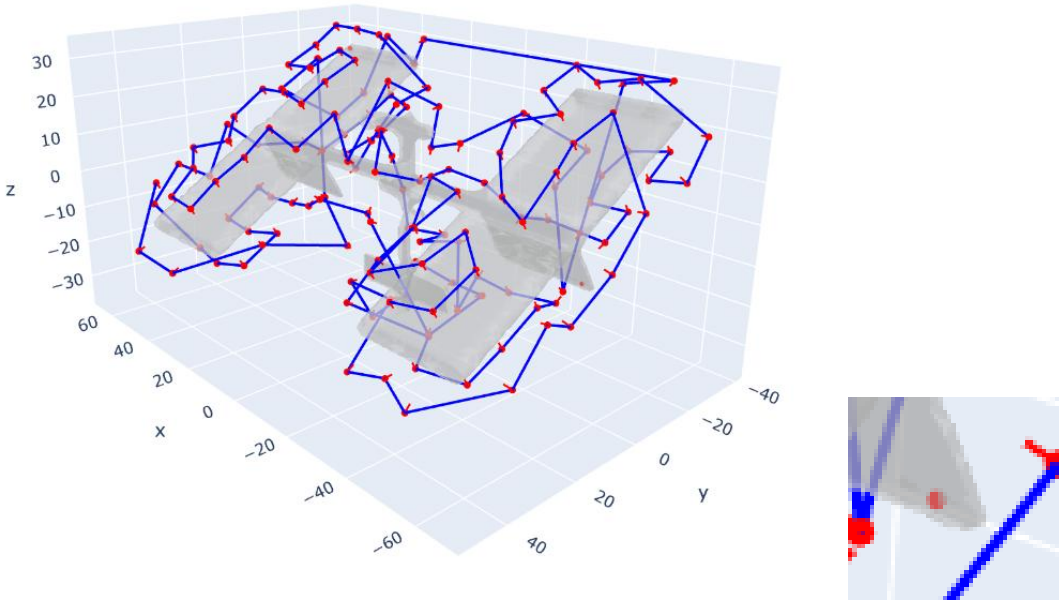

Figure 5.1 Missed coverage plot for the base case, missed face is shown enlarged

5.1.4. Travelling Time & Fuel Consumption

Travelling time is simple to calculate using the pre-existing impulsive dynamics developed for the optimization stage. Fuel consumption, similarly, is easy to calculate, as the delta-v's for the trajectory can be plugged into the rocket equation. As we are using nitrogen thrusters, a specific impulse of approximately 70 can be assumed.

$$\Delta v = I_{sp} g_0 \ln\left(\frac{m_0}{m_f}\right)$$

Where I_{sp} is the specific impulse of the thrusters, g_0 is Earth's gravitational constant, m_0 is the starting mass before a maneuver, and m_f is the final mass after the maneuver.

The base case travel time for a full ISS inspection is 45 minutes, and fuel use is 1.5kg. These are both well within achievable limits. Additionally, given the considerable time required for pre-breathing, donning, and other procedures for current astronauts, the inspection time of 45 minutes offers a vast improvement over astronaut-conducted inspection.

5.2. Simulation

The system is validated in a MuJoCo environment designed to test path-planning command parsing and free-flight control. The simulation setup includes:

- A 25 kg free-floating 0.6 m edge length cubic robot.
- Six face-center-mounted thrusters, each limited to 0-20 N.
- Three orthogonal reaction wheels with torque limits of -5 to 5 Nm.
- A 1000 kg ISS mesh target.

Physics and runtime settings:

- Gravity is disabled.
- Contact is disabled for all reaction wheels to simplify the configuration.

This simulation environment provides an efficient platform for verifying translational and rotational control performance and validating thruster–wheel coordination before extending to higher-fidelity contact or sensing scenarios.

5.3. Anomaly Detection

In live testing, the subsystem reliably identifies:

- Linear cracks down to 1 to 2 mm
- Surface dents and pits
- Abrasion and discoloration regions

Latency on the GTX 1050 Ti averages:

- Preprocessing: 2 to 4 ms
- Inference: 25 to 40 ms
- Postprocessing: 5 to 10 ms
- Total: 32 to 55 ms, yielding 18 to 30 FPS

These results demonstrate that the proposed detection pipeline is well-suited for fast, reliable surface anomaly inspection in real-world robotics and automation environments.

6. Areas to Improve

While our system seems to perform to a high level, there are still some areas on which we would like to improve.

6.1. Collision Detection

Despite our collision detection approach, we still observe some collision cases in the straight line path between waypoints. Our suspicion is that this occurs due to the fact that our mesh is not convex, and possibly because of a misalignment or misinterpretation of face "outward pointing direction". While

this is addressable via manual adjustment of the path, future iterations of the system should be able to produce completely collision free paths.

6.2. Parameter-Dependency

Currently, our pipeline performance is highly dependent on the variables within our pipeline configuration file, which are manually entered and not algorithmically optimized. In particular, the face count and number of cluster variables seem to have a significant impact on the ability of the system to find collision-free, fuel-efficient, and high-coverage trajectories. In future, we would like to find a way to incorporate these pre-processing variables into an optimization function in order to produce a fully optimized pipeline.

6.3. Dynamics

Our current implementation uses free-floating dynamics. This is a valid assumption, given the fact that the CubeSat is very small, conducts low delta-v maneuvers, and conducts missions in timespans less than an hour. However, free-floating dynamics will omit some orbital perturbances that may allow for the satellite trajectory to drift unacceptably, especially if a PD or PiD control scheme is not used. We suggest to shift to Clohessy-Wiltshire in the optimization step to resolve this.

7. Appendix

7.1. Cost & Revenue Analysis

Factor	Value	Notes
Astronaut EVA Hours / Year	140	From NASA documentation
Astronaut Amortized Cost / Hour	$100,000	Derived from NASA documentation
Number of astronauts / EVA	2	Average
Assumed % inspection	0.7	Assumed
Inspection cost / year	$19,600,000	
Assumed annual value of ISS experiments per year	$200,000,000	Derived from NASA research budget and other grants. Includes all NSF and other grants dedicated to ISS research
Assumed lost time % / year	0.1	Assumed
Value lost per year	$20,000,000	
Total cost to NASA per year (total potential revenue for Ark Robotics / year)	$39,600,000	
Expected Platform Cost	$50,000,000	We expect to be able to reach 1/2 the cost of satellite servicing hardware
Payback period	1.262626263	Years

8. Bibliography

[1] B. H. T. A. E. S. L. Apodaca, "In-Orbit Space Structure Inspection," *IEEE Robotics and Automation Letters,* vol. 10, no. 9, pp. 8810-8817, 2025, doi: 10.1109/LRA.2025.3583621.